SYSTEM DESIGN

Banking Business Rules Engine (BRE)

Technical Architecture Diagrams | Flow Diagrams | Sequence Diagrams | Deployment View

Document	System Design — BRE Platform
Version	1.0
Date	June 2026

Diagram 1 — System Context

Shows BRE's position in the overall banking technology landscape and all external system interactions.

Loan Officer (Browser / Mobile)	LOS Service (Spring Boot)	LMS Service (Spring Boot)	CBS (Core Banking)

REST API calls / Kafka events

BRE MICROSERVICE

Rule Mgmt API | Execution Engine | Simulation | Audit Logger | Kafka Consumer

Reads from / Writes to

PostgreSQL (Rule Store + Audit Log)	Redis (Rule Cache + Bureau Cache)	Apache Kafka (Async Events)	S3 / SFTP (Log Archive)

External integrations

CIBIL / Bureau API	KYC / Aadhaar Service	VRM Service	Blacklist / Debarment Svc

Diagram 2 — Synchronous Rule Evaluation Flow

Step-by-step flow when LOS calls BRE for real-time rule evaluation during loan origination.

Step	LOS	API Layer	Execution Svc	Rule Cache	Rule Evaluator	Audit Logger	Kafka
1	POST /evaluate (loan context)						
2		Validate JWT + Schema					
3			Load rules for product	Redis lookup HIT/MISS			
4					Evaluate each rule (SpEL)		
5			Aggregate deviations				
6						Persist exec log (async)	
7							Publish bre.completed
8	Receive EvalResult	Return 200 with results					

Diagram 3 — Rule Lifecycle State Machine

All possible states a rule version can be in, and valid transitions between them.



From State	To State	Triggered By	Side Effects
—	DRAFT	RULE_AUTHOR creates rule	None
DRAFT	IN_REVIEW	Author submits for review	Email notification to RULE_APPROVER
IN_REVIEW	APPROVED	RULE_APPROVER approves	Rule ready for activation
IN_REVIEW	REJECTED	RULE_APPROVER rejects	Author notified with rejection reason
APPROVED	ACTIVE	RULE_APPROVER activates	Previous ACTIVE version -> SUPERSEDED; Redis cache invalidated; Kafka event published
ACTIVE	DEACTIVATED	RULE_APPROVER deactivates	Redis cache invalidated; Kafka event published; rule no longer evaluated
ACTIVE	SUPERSEDED	New version activated	Auto-transition; retained for audit history only

Diagram 4 — Deviation Routing Logic

How multiple deviation rules are aggregated and routed to the correct authority level.

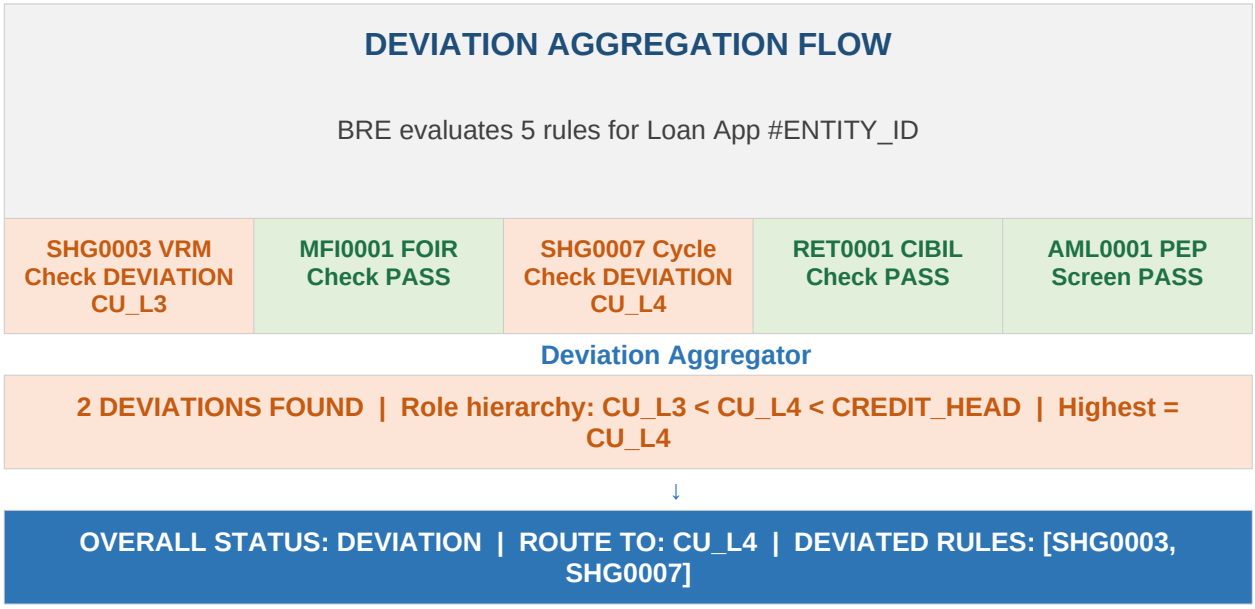


Diagram 5 — Microservice Architecture

Shows all microservices in the Lending Origination System ecosystem and how BRE fits and communicates with each.

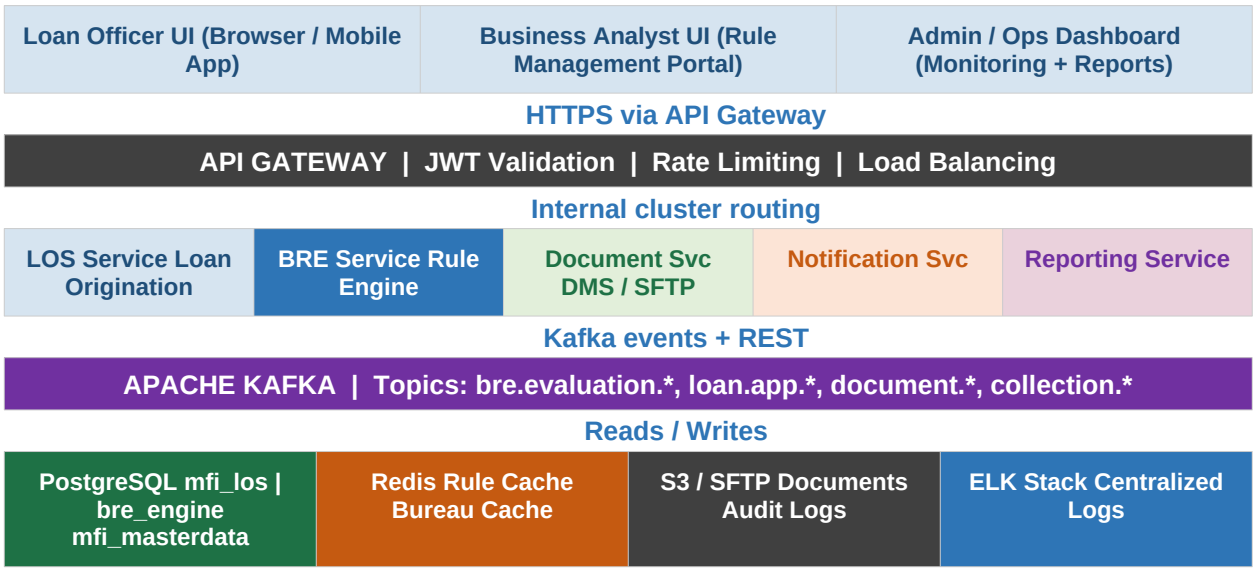


Diagram 6 — Kubernetes Deployment View

Production deployment topology showing pods, replicas, and infrastructure components.

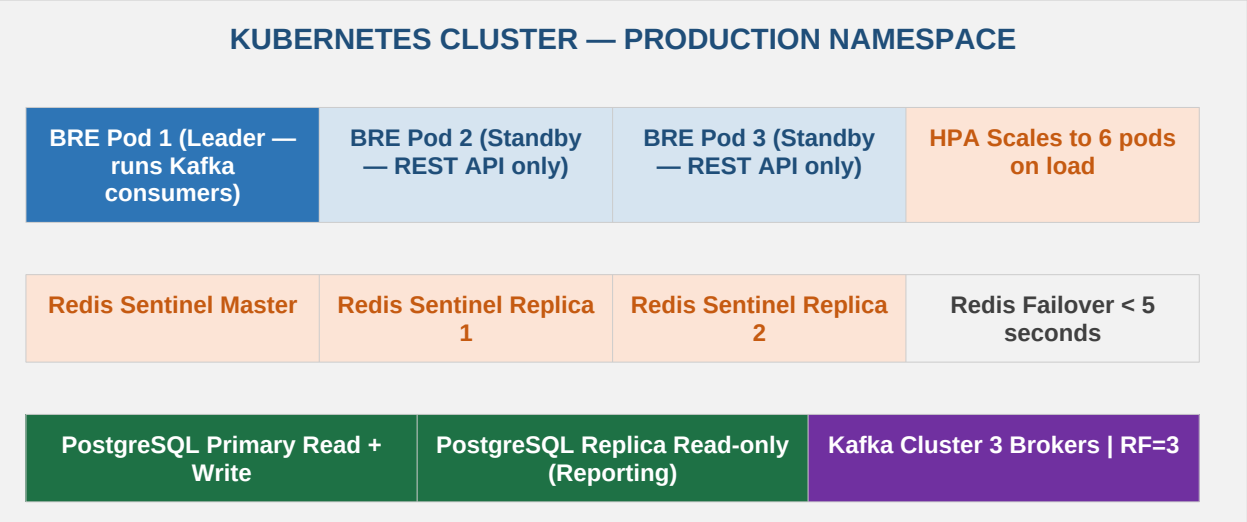


Diagram 7 — Cache-Aside Pattern for Rule Loading

How BRE loads rules on each evaluation request using the cache-aside pattern.

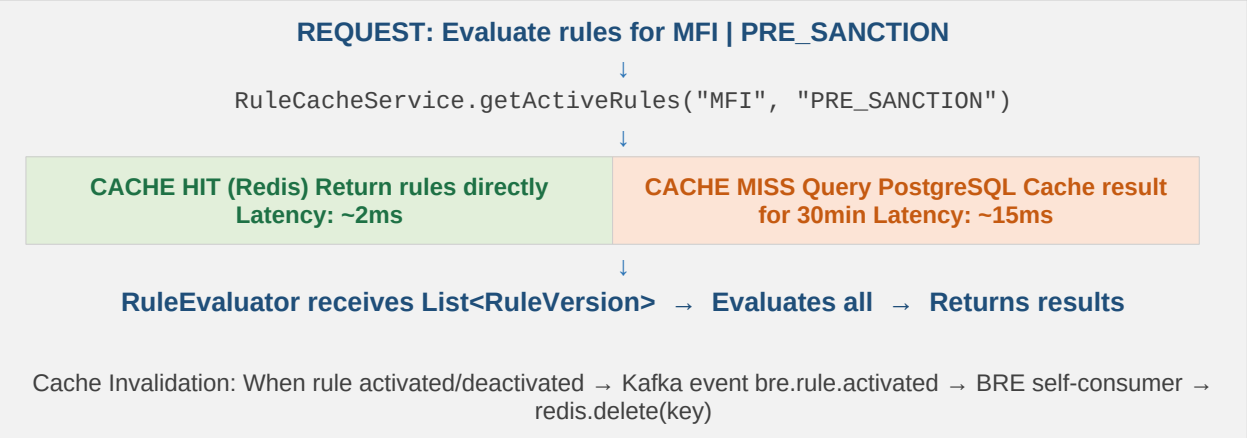


Diagram 8 — End-to-End Loan Journey with BRE

Complete loan origination journey showing every BRE touchpoint from application to disbursement.

Stage	LOS Action	BRE Rules Evaluated	BRE Outcome
1	Borrower KYC + profile creation	AML0001 — PEP/Sanctions screening RET0005 — Blacklist check	PASS → proceed FAIL → reject application immediately
2	Group/SHG verification	SHG0001 — Group formation SHG0007 — Cycle type validation	PASS → proceed to credit FAIL → return to field officer
3	Credit evaluation request	MFI0001 — FOIR check MFI0002 — Active loan count RET0001 — CIBIL threshold	PASS → auto-approve DEVIATION → route to CU_L3/L4 FAIL → reject
4	VRM + Location check	SHG0003 — VRM category check	A/B: PASS C: DEVIATION → CU_L3 D/E: FAIL
5	Loan amount / product eligibility	RET0002 — LTV ratio (if secured) SME0004 — Sector exposure (SME) RET0003 — EMI/NMI ratio	PASS → proceed to sanction DEVIATION → route to CREDIT_HEAD
6	Pre-disbursement check	Re-run all ACTIVE rules (conditions may have changed)	All PASS → disburse Any FAIL → hold disbursement

9. Scaling Strategy

Bottleneck	Strategy	Detail
High evaluation RPS	Horizontal scaling	BRE pods are fully stateless — HPA adds pods on CPU > 70%
Rule load latency	Redis cache-aside	Active rules cached 30 min; DB hit only on cold start or cache miss
Bureau API slowness	Response caching + circuit breaker	CIBIL responses cached 24h per PAN; Resilience4j CB after 3 failures
Audit log write volume	Async write + table partitioning	Audit logged async via internal queue; monthly partitioned PostgreSQL table
Kafka consumer lag	Leader election	Only 1 node runs all 50+ consumers via Redis SET NX PX leader election